In [4]:
```
import pandas as pd
from pathlib import Path

file_path = Path(
    r"C:\Users\red devil\Desktop\PROJECT BI\P4\New folder\Retailfixed.csv"
)

column_names = [
    "Invoice", "StockCode", "Description", "Quantity",
    "InvoiceDate", "Price", "Customer_ID", "Country", "SourceSheet"
]

df = pd.read_csv(
    file_path,
    header=None,
    names=column_names,
    low_memory=False,
    dtype={"Invoice": "string", "StockCode": "string"}
)

df.columns = df.columns.str.strip().str.replace(" ", "_", regex=False)

print(df.columns.tolist())
print(df.head(3))
```

```
['Invoice', 'StockCode', 'Description', 'Quantity', 'InvoiceDate', 'Price', 'Custome
r_ID', 'Country', 'SourceSheet']
  Invoice StockCode                          Description  Quantity  \
0  489434     21232       STRAWBERRY CERAMIC TRINKET BOX        24
1  489434     21523  FANCY FONT HOME SWEET HOME DOORMAT        10
2  489434     21871                SAVE THE PLANET MUG        24

          InvoiceDate  Price  Customer_ID         Country SourceSheet
0  2009-12-01T07:45:00   1.25      13085.0  United Kingdom   2010-2011
1  2009-12-01T07:45:00   5.95      13085.0  United Kingdom   2010-2011
2  2009-12-01T07:45:00   1.25      13085.0  United Kingdom   2010-2011
```

In [5]:
```
df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"], errors="coerce")

print("Unparsed dates:", df["InvoiceDate"].isna().sum())
print("Date range:", df["InvoiceDate"].min(), "to", df["InvoiceDate"].max())
```

```
Unparsed dates: 0
Date range: 2009-12-01 07:45:00 to 2011-12-09 12:50:00
```

In [6]:
```
# Normalize IDs and numeric fields
df["Customer_ID"] = (
    pd.to_numeric(df["Customer_ID"], errors="coerce")
    .astype("Int64")
    .astype("string")
)
df["Quantity"] = pd.to_numeric(df["Quantity"], errors="coerce")
df["Price"] = pd.to_numeric(df["Price"], errors="coerce")
df["LineRevenue"] = df["Quantity"] * df["Price"]
```

```
has_customer = df["Customer_ID"].notna()
is_cancelled = df["Invoice"].str.upper().str.startswith("C", na=False)

# Completed purchases determine frequency and last purchase date
purchases = df[
    has_customer
    & ~is_cancelled
    & df["Quantity"].gt(0)
    & df["Price"].gt(0)
].copy()

# Include negative lines in monetary value so returns/cancellations reduce revenue
revenue_rows = df[has_customer].copy()

frequency = (
    purchases.groupby("Customer_ID")["Invoice"]
    .nunique()
    .rename("Frequency")
)

last_purchase = (
    purchases.groupby("Customer_ID")["InvoiceDate"]
    .max()
    .rename("LastPurchaseDate")
)

monetary = (
    revenue_rows.groupby("Customer_ID")["LineRevenue"]
    .sum()
    .rename("Monetary")
)

rfm = pd.concat([last_purchase, frequency, monetary], axis=1).reset_index()
rfm["Frequency"] = rfm["Frequency"].fillna(0).astype(int)

snapshot_date = df["InvoiceDate"].max().normalize() + pd.Timedelta(days=1)
rfm["Recency"] = (
    snapshot_date - rfm["LastPurchaseDate"].dt.normalize()
).dt.days

print("Snapshot date:", snapshot_date.date())
print("Customers with purchase history:", len(rfm))
rfm.head()
```

```
Snapshot date: 2011-12-10
Customers with purchase history: 5942
```

Out[6]:

	Customer_ID	LastPurchaseDate	Frequency	Monetary	Recency
0	12346	2011-01-18 10:01:00	12	-51.74	326.0
1	12347	2011-12-07 15:52:00	8	4921.53	3.0
2	12348	2011-09-25 13:13:00	5	2019.40	76.0
3	12349	2011-11-21 09:51:00	4	4404.54	19.0
4	12350	2011-02-02 16:01:00	1	334.40	311.0

In [7]:
```
print("Customers with no valid purchase date:", rfm["LastPurchaseDate"].isna().sum(

# RFM scores require at least one completed purchase
rfm = rfm.dropna(subset=["LastPurchaseDate", "Recency"]).copy()

print("Customers included in RFM:", len(rfm))
```

```
Customers with no valid purchase date: 64
Customers included in RFM: 5878
```

In [8]:
```
# Create RFM scores
rfm["R_Score"] = pd.qcut(
    rfm["Recency"].rank(method="first"),
    5,
    labels=[5, 4, 3, 2, 1]
).astype(int)

rfm["F_Score"] = pd.qcut(
    rfm["Frequency"].rank(method="first"),
    5,
    labels=[1, 2, 3, 4, 5]
).astype(int)

rfm["M_Score"] = pd.qcut(
    rfm["Monetary"].rank(method="first"),
    5,
    labels=[1, 2, 3, 4, 5]
).astype(int)

# Flag customers with no purchase in the last 90 days
rfm["IsChurned"] = rfm["Recency"] > 90

# Assign a business segment
def segment_customer(row):
    if row["Recency"] > 90:
        return "Lost"
    elif row["R_Score"] >= 4 and row["F_Score"] >= 4 and row["M_Score"] >= 4:
        return "Champions"
    elif row["R_Score"] >= 3 and row["F_Score"] >= 3:
        return "Loyal"
    elif row["R_Score"] <= 2 and row["F_Score"] >= 3:
        return "At Risk"
    elif row["R_Score"] >= 4 and row["F_Score"] <= 2:
        return "New"
```

```
    else:
        return "Needs Attention"

rfm["Segment"] = rfm.apply(segment_customer, axis=1)

summary = (
    rfm.groupby("Segment")
       .agg(
           Customers=("Customer_ID", "count"),
           NetMonetary=("Monetary", "sum"),
           ChurnedCustomers=("IsChurned", "sum")
       )
       .sort_values("NetMonetary", ascending=False)
)

print(summary)
```

Segment	Customers	NetMonetary	ChurnedCustomers
Champions	1290	11396998.70	0
Lost	2989	3025670.08	2989
Loyal	971	1605424.88	0
New	441	220858.26	0
Needs Attention	187	102089.02	0

In [9]:
```
summary = (
    rfm.groupby("Segment")
       .agg(
           Customers=("Customer_ID", "count"),
           NetMonetary=("Monetary", "sum"),
           ChurnedCustomers=("IsChurned", "sum")
       )
)

summary["CustomerSharePct"] = (
    summary["Customers"] / summary["Customers"].sum() * 100
)

summary["RevenueSharePct"] = (
    summary["NetMonetary"] / summary["NetMonetary"].sum() * 100
)

summary = summary.sort_values("NetMonetary", ascending=False)

output_dir = file_path.parent / "python_outputs"
output_dir.mkdir(exist_ok=True)

rfm.to_csv(output_dir / "customer_rfm.csv", index=False)
summary.to_csv(output_dir / "segment_summary.csv")

print(summary.round(2))
print("Saved files in:", output_dir)
```

```
                 Customers  NetMonetary  ChurnedCustomers  CustomerSharePct  \
Segment
Champions             1290  11396998.70                 0             21.95
Lost                  2989   3025670.08              2989             50.85
Loyal                  971   1605424.88                 0             16.52
New                    441    220858.26                 0              7.50
Needs Attention        187    102089.02                 0              3.18

                 RevenueSharePct
Segment
Champions                  69.70
Lost                       18.50
Loyal                       9.82
New                         1.35
Needs Attention             0.62
Saved files in: C:\Users\red devil\Desktop\PROJECT BI\P4\New folder\python_outputs
```

In [10]:

```python
# Monthly sales and customer activity
monthly_revenue = (
    revenue_rows.assign(
        InvoiceMonth=revenue_rows["InvoiceDate"].dt.to_period("M").astype(str)
    )
    .groupby("InvoiceMonth")
    .agg(NetRevenue=("LineRevenue", "sum"))
)

monthly_activity = (
    purchases.assign(
        InvoiceMonth=purchases["InvoiceDate"].dt.to_period("M").astype(str)
    )
    .groupby("InvoiceMonth")
    .agg(
        Orders=("Invoice", "nunique"),
        ActiveCustomers=("Customer_ID", "nunique"),
        UnitsSold=("Quantity", "sum")
    )
)

monthly_summary = monthly_revenue.join(monthly_activity, how="outer").reset_index()

# Country sales and activity
country_revenue = (
    revenue_rows.groupby("Country")
    .agg(NetRevenue=("LineRevenue", "sum"))
)

country_activity = (
    purchases.groupby("Country")
    .agg(
        Orders=("Invoice", "nunique"),
        Customers=("Customer_ID", "nunique"),
        UnitsSold=("Quantity", "sum")
    )
)

country_summary = (
```

```
    country_revenue.join(country_activity, how="left")
    .sort_values("NetRevenue", ascending=False)
    .reset_index()
)

monthly_summary.to_csv(output_dir / "monthly_summary.csv", index=False)
country_summary.to_csv(output_dir / "country_summary.csv", index=False)

print(monthly_summary.head())
print(country_summary.head(10))
```

```
  InvoiceMonth  NetRevenue  Orders  ActiveCustomers  UnitsSold
0      2009-12   660125.10    1512              955     398660
1      2010-01   530436.51    1011              720     370080
2      2010-02   487596.42    1104              772     371855
3      2010-03   633419.31    1524             1057     502099
4      2010-04   558007.83    1329              942     350585
          Country   NetRevenue  Orders  Customers  UnitsSold
0  United Kingdom  13482505.07   33541       5350    8532394
1            EIRE    573509.76     567          5     318021
2     Netherlands    548330.70     228         22     383879
3         Germany    411959.16     789        107     225153
4          France    320046.26     614         95     270288
5       Australia    166444.42      95         15     103759
6     Switzerland     98779.41      90         22      52227
7           Spain     91013.44     154         41      50307
8          Sweden     87421.52     104         19      88495
9         Denmark     64459.59      43         12     237471
```

In [11]:
```
eire_customers = (
    purchases[purchases["Country"].eq("EIRE")]
    .groupby("Customer_ID")
    .agg(
        Orders=("Invoice", "nunique"),
        Units=("Quantity", "sum"),
        Revenue=("LineRevenue", "sum")
    )
    .sort_values("Revenue", ascending=False)
)

print(eire_customers)
```

```
             Orders   Units    Revenue
Customer_ID
14156           156  164325  313437.62
14911           398  147972  291420.81
14016            11    5722   11403.41
12745             1       1     201.56
16047             1       1     107.14
```

In [12]:
```
country_summary["RevenueSharePct"] = (
    country_summary["NetRevenue"] / country_summary["NetRevenue"].sum() * 100
)

print("Top revenue months:")
print(monthly_summary.nlargest(5, "NetRevenue").to_string(index=False))
```

```
print("\nLowest revenue months:")
print(monthly_summary.nsmallest(5, "NetRevenue").to_string(index=False))

print("\nTop countries by revenue share:")
print(
    country_summary[
        ["Country", "NetRevenue", "RevenueSharePct", "Orders", "Customers"]
    ].head(10).round(2).to_string(index=False)
)
```

```
Top revenue months:
InvoiceMonth  NetRevenue  Orders  ActiveCustomers  UnitsSold
     2010-11  1129025.16    2587             1607     653066
     2011-11  1126815.07    2657             1664     665923
     2011-10   973306.38    1929             1364     591543
     2010-10   961520.74    2133             1497     596432
     2011-09   929356.23    1755             1266     543650

Lowest revenue months:
InvoiceMonth  NetRevenue  Orders  ActiveCustomers  UnitsSold
     2011-12   341557.43     778              615     286778
     2011-04   425222.67    1149              856     291365
     2011-02   435534.07     997              758     265027
     2011-01   473731.90     987              741     348473
     2010-02   487596.42    1104              772     371855

Top countries by revenue share:
       Country  NetRevenue  RevenueSharePct  Orders  Customers
United Kingdom 13482505.07            82.77   33541       5350
          EIRE   573509.76             3.52     567          5
   Netherlands   548330.70             3.37     228         22
       Germany   411959.16             2.53     789        107
        France   320046.26             1.96     614         95
     Australia   166444.42             1.02      95         15
   Switzerland    98779.41             0.61      90         22
         Spain    91013.44             0.56     154         41
        Sweden    87421.52             0.54     104         19
       Denmark    64459.59             0.40      43         12
```

In [13]:

```
import matplotlib.pyplot as plt
import pandas as pd

monthly_plot = monthly_summary.copy()
monthly_plot["InvoiceMonth"] = pd.to_datetime(monthly_plot["InvoiceMonth"])

country_plot = country_summary.nlargest(10, "NetRevenue").sort_values("NetRevenue")
segment_plot = summary.sort_values("RevenueSharePct")

fig, axes = plt.subplots(3, 1, figsize=(12, 15))

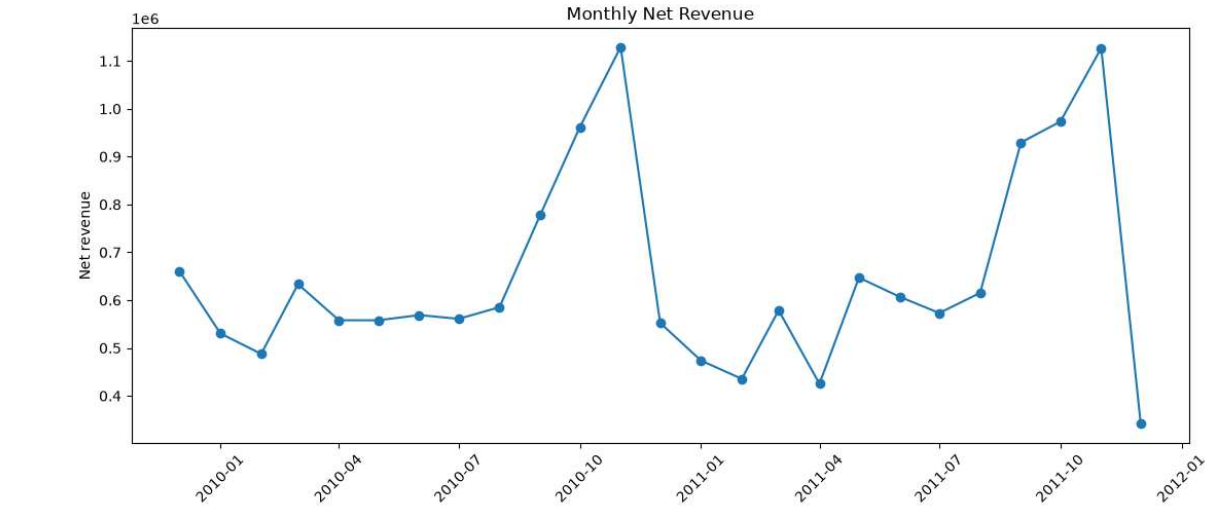
axes[0].plot(monthly_plot["InvoiceMonth"], monthly_plot["NetRevenue"], marker="o")
axes[0].set_title("Monthly Net Revenue")
axes[0].set_ylabel("Net revenue")
axes[0].tick_params(axis="x", rotation=45)

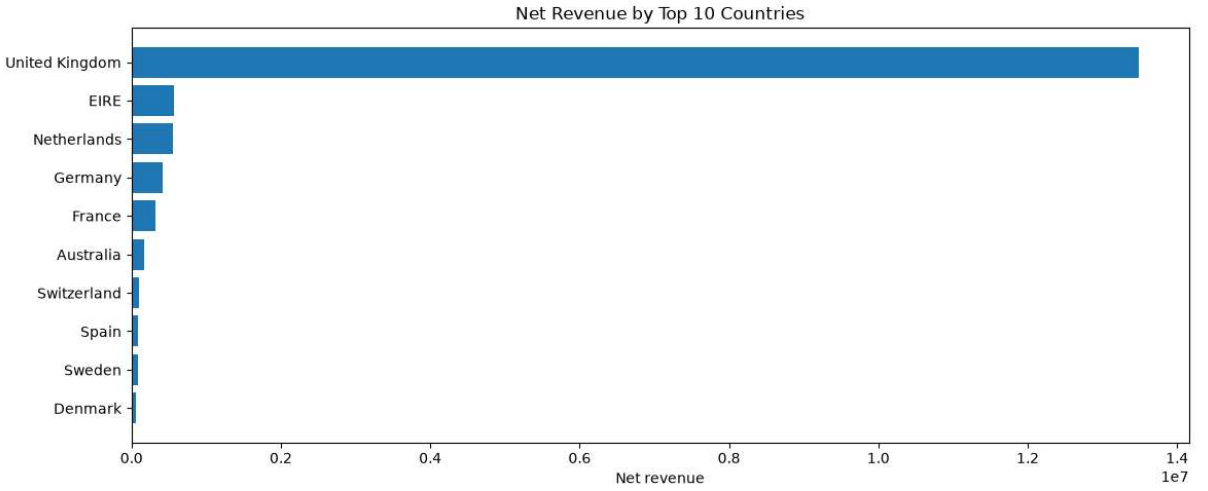
axes[1].barh(country_plot["Country"], country_plot["NetRevenue"])
```

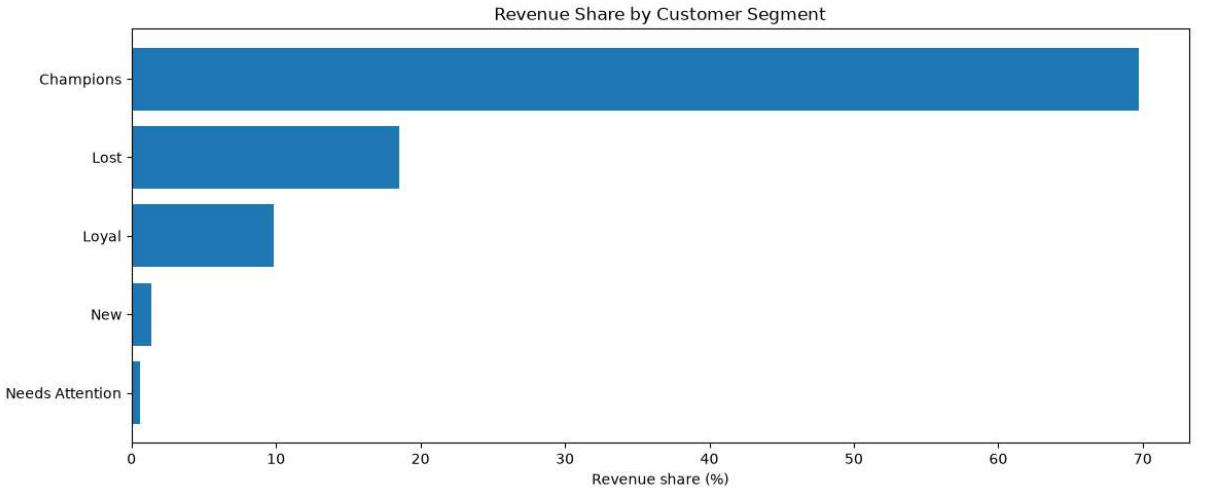
```
axes[1].set_title("Net Revenue by Top 10 Countries")
axes[1].set_xlabel("Net revenue")

axes[2].barh(segment_plot.index, segment_plot["RevenueSharePct"])
axes[2].set_title("Revenue Share by Customer Segment")
axes[2].set_xlabel("Revenue share (%)")

plt.tight_layout()
plt.savefig(output_dir / "analysis_charts.png", dpi=150, bbox_inches="tight")
plt.show()
```







In [16]:
```
from pathlib import Path
import pandas as pd

# 1. Define output_dir (this creates a folder named 'output' automatically if it do
output_dir = Path(".")
output_dir.mkdir(parents=True, exist_ok=True)

# 2. Process transactions
transactions = df.copy()
transactions["Revenue"] = transactions["Quantity"] * transactions["Price"]
transactions["InvoiceMonth"] = transactions["InvoiceDate"].dt.to_period("M").astype

# 3. Save to CSV
transaction_file = output_dir / "transactions_for_powerbi.csv"
transactions.to_csv(transaction_file, index=False)

print("Saved:", transaction_file.resolve())
print("Exists:", transaction_file.exists())
```

```
Saved: C:\Users\red devil\Desktop\PROJECT BI\P4\New folder\transactions_for_powerbi.
csv
Exists: True
```

In [ ]:

In [ ]: